ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Higher-order challenge II

Work in groups of 3–4. For each challenge, discuss your approach before coding.
Write your solutions in Haskell / Scala / Elixir as requested.
When you're confident in a solution, test it in with relevant interpreter.

Warm-up

Challenge 1. Point-free workout

A) Using *only* partial application and composition (`.`), and without writing any lambdas, define:

```
negateAll :: [Int] -> [Int]        -- negate every element
longWords :: [String] -> [String]  -- keep strings with length > 4
countEven :: [Int] -> Int          -- count how many elements are even
```

Examples:

```
negateAll [1, -2, 3]          == [-1, 2, -3]
longWords ["hi", "hello", "world", "ok"] == ["hello", "world"]
countEven [1, 2, 3, 4, 5, 6] == 3
```

Key decisions / observations:

- For each function, which higher-order operation is the right one — `map` , `filter` , or something else?
- Can you express the argument to that operation without a lambda `\x -> ...` , using only *operator sections* (e.g. `(* 2)` , `(> 5)`) and *function composition* (`.`)?

B) *Scala alternative:* try to translate your point-free Haskell definitions into Scala. Where does the translation break down? What do you have to fall back on?

Challenge 2. Flip at work

The function `flip :: (a -> b -> c) -> b -> a -> c` swaps the first two arguments.

Without looking at the notes, write the definition of `flip`. Then use it to define:

```
prepend :: [a] -> a -> [a]    -- prepend x xs = xs ++ [x]  (but using flip)
```

Key decision / observation:

- which standard function, when flipped, gives you `prepend`?

The `reverse` function on lists can be defined as follows with `foldr`

```
reverse = foldr (\x acc -> acc ++ [x]) []
```

Use `flip` to write a similarly concise definition of `reverse` using `foldl`.

Core

Challenge 3. Lifting

A) An *arithmetic lifter* takes a binary operation on numbers and "lifts" it to work on functions. Define:

```
lift :: (b -> b -> b) -> (a -> b) -> (a -> b) -> (a -> b)
```

such that `lift (+) f g` returns a function that, given `x`, computes `f x + g x`.

Then use `lift` to define:

```
adder, multiplier :: (Double -> Double) -> (Double -> Double) -> (Double-> Double)
```

Test: `lift (+) (*3.0) (*5.0) $ 2 == 16.0` (i.e. `6.0 + 10.0`)

Key decisions / observations

- What does it mean to *add* two functions? If `f` and `g` both take an `Int` and return a `Double`, what should `f + g` do with a given input?

- Is there anything specific to `(+)` in your answer, or would the same idea work with `(*)`, `(&&)`, `max`, or any other binary operation?

B) *Elixir alternative:* define the same `lift` function in Elixir. Since Elixir has no type signatures, start from the intent: `lift` takes an operator and two functions and returns a function.

```
lift = fn op, f, g -> ...
```

Then provide an Elixir test expression corresponding to the Haskell expression above.

Key decisions / observations

- How does the definition compare with Haskell's? Is it simpler or harder to read without the types?
- In Haskell, `(+)` turns an operator into a function you can pass as argument. How do you pass `+` as an argument in Elixir? What does this tell you about the status of operators in the two languages?

Challenge 4. Meet and meetAll

A) Using `lift` from Challenge 3, define a lifted version of the Boolean `(&&)` operator:

```
meet :: (Int -> Bool) -> (Int -> Bool) -> (Int -> Bool)
```

such that `meet f g` returns a predicate that is true when *both* `f` and `g` are true.

Now generalize `lift` to accept a *list* of functions:

```
liftAll :: (b -> b -> b) -> b -> [a -> b] -> (a -> b)
```

such that `liftAll (+) 0 [f1, f2, f3]` returns a function that sums the results of all three on any input.

Key decision:

- what tool combines a list of things using a binary operation and a base value?

Use `liftAll` to define:

```
meetAll :: [Int -> Bool] -> (Int -> Bool)
```

Test: `meetAll [even, (>0), (<100)] 42 == True`

B) *Elixir alternative:* define `meet_all` in Elixir.

Key observation

- In Elixir, anonymous functions are called with a dot (`f.(x)`), and `Enum.reduce` plays the role of `foldr` . How does this affect the definition?

Advanced

Challenge 5. Church numerals

In the lambda calculus, natural numbers are represented as *iterators*: the number n is the function that applies its argument n times. We define:

```
type Church a = (a -> a) -> a -> a
```

A value of type `Church a` takes a function `f :: a -> a` and a starting value `x :: a` , and applies `f` some number of times to `x` . The number of applications *is* the numeral.

Now define:

```
zero, one, two, three :: Church a

toInt   :: Church Int -> Int
fromInt :: Int -> Church a

churchAdd :: Church a -> Church a -> Church a
churchMul :: Church a -> Church a -> Church a
```

Key decisions:

- `zero f x = ?` — what does it mean to apply `f` zero times?
- `one f x = ?`
- For `toInt` : what function do you iterate, and what do you start from?
- For `churchAdd` : if `m` applies `f` m-times and `n` applies it n-times, how do you get m+n applications?
- For `churchMul` : if `n f` is "apply `f` n-times", what does `m (n f)` give you?

Test your definitions:

```
toInt (churchAdd two three)  == 5
toInt (churchMul two three)  == 6
```

Bonus (hard): define `churchExp :: Church a -> Church (a -> a) -> Church a` such that `churchExp m n` represents m^n. Why does `n` need the type `Church (a -> a)` rather than `Church a`? Test: `toInt (churchExp two (fromInt 3)) == 8`.

Wrap-up (15 min)

Each group presents one of the following to the class (choose one):

- **The trickiest challenge you solved** — walk through your reasoning.
- **A challenge you couldn't solve** — explain where you got stuck and what you tried. (This is just as valuable.)
- **A surprise** — something you discovered that wasn't in the lecture notes.